Big Data Analytics Lab — All Programs & Commands

TE / V (A & B) • Academic Year 2026–27 • Subject Code: CEPEL5014

Mid Term Practical Examination — Complete Preparation PDF
This document contains all the programs/commands provided for the six experiments in the uploaded Big Data Analytics Lab experiment list. The experiment list specifies HDFS, MapReduce Word Count, MongoDB/HBase, Hive descriptive analytics, one data-stream algorithm, and visualization.

Experiment 1 — Hadoop HDFS Practical

The experiment list covers HDFS basics, Hadoop ecosystem overview, installing Hadoop, copying files to Hadoop, copying/deleting files, and moving/displaying files in HDFS.

A. Check Hadoop installation

```
hadoop version
```

B. Create directory in HDFS

```
hdfs dfs -mkdir /bda
```

C. Create subdirectory

```
hdfs dfs -mkdir /bda/input
```

D. Display directories/files

```
hdfs dfs -ls /
hdfs dfs -ls /bda
```

E. Copy local file to HDFS

Suppose `data.txt` is present in your current folder:

```
hdfs dfs -put data.txt /bda/input/
```

Alternative command:

```
hdfs dfs -copyFromLocal data.txt /bda/input/
```

F. Display file contents

```
hdfs dfs -cat /bda/input/data.txt
```

G. Copy file from HDFS to local system

```
hdfs dfs -get /bda/input/data.txt .
```

Alternative command:

```
hdfs dfs -copyToLocal /bda/input/data.txt .
```

H. Move file in HDFS

```
hdfs dfs -mv /bda/input/data.txt /bda/
```

I. Delete file

```
hdfs dfs -rm /bda/data.txt
```

J. Delete directory

```
hdfs dfs -rm -r /bda/input
```

Most important HDFS commands

```
hdfs dfs -mkdir
```

```
hdfs dfs -ls
hdfs dfs -put
hdfs dfs -get
hdfs dfs -cat
hdfs dfs -mv
hdfs dfs -cp
hdfs dfs -rm
hdfs dfs -rm -r
```

Experiment 2 — Hadoop MapReduce: Word Count

The experiment asks for a program to implement a word count program using MapReduce.

WordCount.java — complete program

```
import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

    // Mapper
    public static class TokenizerMapper
            extends Mapper<Object, Text, Text, IntWritable> {

        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(Object key, Text value, Context context)
                throws IOException, InterruptedException {

            String[] words = value.toString().split("\\s+");

            for (String w : words) {
                word.set(w);
                context.write(word, one);
            }
        }
    }

    // Reducer
    public static class IntSumReducer
            extends Reducer<Text, IntWritable, Text, IntWritable> {

        public void reduce(Text key, Iterable<IntWritable> values,
                           Context context)
                throws IOException, InterruptedException {

            int sum = 0;

            for (IntWritable value : values) {
                sum += value.get();
            }

            context.write(key, new IntWritable(sum));
        }
    }

    // Driver
    public static void main(String[] args)
            throws Exception {

        Configuration conf = new Configuration();

        Job job = Job.getInstance(conf, "word count");

        job.setJarByClass(WordCount.class);

        job.setMapperClass(TokenizerMapper.class);
        job.setReducerClass(IntSumReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
```

```
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Input — input.txt

```
hello world
hello hadoop
hadoop world
```

Expected output

```
hadoop  2
hello   2
world   2
```

MapReduce flow

```
Input
  ↓
Mapper
  ↓
<word, 1>
  ↓
Shuffle & Sort
  ↓
Reducer
  ↓
<word, count>
```

Experiment 3 — MongoDB NoSQL Practical

The experiment list specifies installing/configuring MongoDB or HBase and executing NoSQL commands. The following section contains the MongoDB command set provided for preparation.

Start MongoDB shell

```
mongosh
```

Create/use database

```
use bda
```

Create collection

```
db.createCollection("students")
```

Insert one document

```
db.students.insertOne({
    name: "Vaishnavi",
    age: 20,
    branch: "Computer",
    marks: 85
})
```

Insert multiple documents

```
db.students.insertMany([
    {
        name: "Amit",
        age: 21,
        branch: "Computer",
        marks: 78
    },
    {
        name: "Priya",
        age: 20,
        branch: "IT",
        marks: 92
    },
    {
        name: "Rahul",
        age: 22,
        branch: "Computer",
        marks: 67
    }
])
```

Display all records

```
db.students.find()
```

Pretty format:

```
db.students.find().pretty()
```

Find specific record

```
db.students.find({branch: "Computer"})
```

Find marks greater than 80

```
db.students.find({marks: {$gt: 80}})
```

Find marks less than 80

```
db.students.find({marks: {$lt: 80}})
```

Update

```
db.students.updateOne(
    {name: "Amit"},
    {$set: {marks: 88}}
)
```

Delete

```
db.students.deleteOne({name: "Rahul"})
```

Count documents

```
db.students.countDocuments()
```

Sort — ascending

```
db.students.find().sort({marks: 1})
```

Sort — descending

```
db.students.find().sort({marks: -1})
```

Drop collection

```
db.students.drop()
```

Experiment 4 — Hive Database & Descriptive Analytics

The experiment asks for creation of a Hive database and descriptive analytics/basic statistics.

Create database

```
CREATE DATABASE bda;
```

Show databases

```
SHOW DATABASES;
```

Select database

```
USE bda;
```

Create table

```
CREATE TABLE students (
    id INT,
    name STRING,
    age INT,
    marks FLOAT
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ',';
```

Insert data

```
INSERT INTO students VALUES
(1,'Amit',21,78),
(2,'Priya',20,92),
(3,'Rahul',22,67),
(4,'Neha',21,85),
(5,'Riya',20,95);
```

Display data

```
SELECT * FROM students;
```

Descriptive Statistics — Count

```
SELECT COUNT(*) FROM students;
```

Average

```
SELECT AVG(marks) FROM students;
```

Maximum

```
SELECT MAX(marks) FROM students;
```

Minimum

```
SELECT MIN(marks) FROM students;
```

Sum

```
SELECT SUM(marks) FROM students;
```

Standard deviation

```
SELECT STDDEV(marks) FROM students;
```

Variance

```
SELECT VARIANCE(marks) FROM students;
```

Complete basic statistics — important

```
SELECT
    COUNT(marks) AS count,
    AVG(marks) AS average,
    MIN(marks) AS minimum,
```

```
    MAX(marks) AS maximum,
    STDDEV(marks) AS standard_deviation,
    VARIANCE(marks) AS variance
FROM students;
```

Experiment 5 — Data Stream Algorithm: Bloom Filter

The experiment allows any one of DGIM, Bloom Filter, or Flajolet-Martin. This PDF includes the Bloom Filter program provided for preparation.

Python Program — Bloom Filter

```
import hashlib

class BloomFilter:

    def __init__(self, size):
        self.size = size
        self.bit_array = [0] * size

    def hash1(self, item):
        return int(hashlib.md5(item.encode()).hexdigest(), 16) % self.size

    def hash2(self, item):
        return int(hashlib.sha1(item.encode()).hexdigest(), 16) % self.size

    def add(self, item):
        index1 = self.hash1(item)
        index2 = self.hash2(item)

        self.bit_array[index1] = 1
        self.bit_array[index2] = 1

    def check(self, item):
        index1 = self.hash1(item)
        index2 = self.hash2(item)

        return (
            self.bit_array[index1] == 1 and
            self.bit_array[index2] == 1
        )


# Create Bloom Filter
bf = BloomFilter(10)

# Add elements
bf.add("apple")
bf.add("banana")
bf.add("mango")

# Check elements
print("apple:", bf.check("apple"))
print("banana:", bf.check("banana"))
print("orange:", bf.check("orange"))
```

Expected output

```
apple: True
banana: True
orange: False
```

Bloom Filter reminder

```
Definitely NOT present
        OR
Maybe present
```

> It can have false positives, but not false negatives.

Experiment 6 — Data Visualization

The experiment list permits data visualization using Hive/PIG/R/Tableau. The following Python visualization programs are the programs provided for preparation.

Python — Bar Chart

```
import matplotlib.pyplot as plt

students = ["Amit", "Priya", "Rahul", "Neha", "Riya"]
marks = [78, 92, 67, 85, 95]

plt.bar(students, marks)

plt.xlabel("Students")
plt.ylabel("Marks")
plt.title("Student Marks")

plt.show()
```

Python — Line Chart

```
import matplotlib.pyplot as plt

students = ["Amit", "Priya", "Rahul", "Neha", "Riya"]
marks = [78, 92, 67, 85, 95]

plt.plot(students, marks, marker='o')

plt.xlabel("Students")
plt.ylabel("Marks")
plt.title("Student Marks")

plt.show()
```

Python — Pie Chart

```
import matplotlib.pyplot as plt

students = ["Amit", "Priya", "Rahul", "Neha", "Riya"]
marks = [78, 92, 67, 85, 95]

plt.pie(marks, labels=students, autopct='%1.1f%%')

plt.title("Student Marks Distribution")

plt.show()
```

Quick practical priority

■■■■■ **HDFS:** all hdfs dfs commands
■■■■■ **MapReduce:** Word Count Java program
■■■■■ **MongoDB:** CRUD commands
■■■■■ **Hive:** create table + statistics queries
■■■■ **Bloom Filter:** complete Python program
■■■ **Visualization:** bar/line/pie chart

Source note

The uploaded experiment-list PDF is the basis for the six experiment headings and required topics. The code and command examples in this document reproduce all of the code/commands supplied in the preceding response; where the source experiment list did not prescribe a particular implementation, the implementation is presented as practical preparation material rather than as text taken from the source.